
ΚΑΤΑΝΕΜΗΜΕΝΑ ΣΥΣΤΗΜΑΤΑ

1^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ:

Υλοποίηση εφαρμογής συνομιλιών βασισμένη στο
μοντέλο πελάτη-εξυπηρετητή

ΟΜΑΔΑ 5380-5226

Κωνσταντίνος Ζώης, ΑΜ: 5226, e-mail: cs05226@uoi.gr

Ιωάννης Φίλλης, ΑΜ: 5380, e-mail: cs05380@uoi.gr

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

ΑΠΡΙΛΙΟΣ 2026

1 ΕΙΣΑΓΩΓΗ

Στην παρούσα άσκηση καλούμαστε να υλοποιήσουμε μια απλή εφαρμογή συνομιλιών, η οποία βασίζεται στο μοντέλο πελάτη-εξυπηρετητή. Μας δίνεται έτοιμη μια αρχική υλοποίηση σε αρχεία γλώσσας C, στα οποία βρίσκεται μια πρώτη έκδοση του πελάτη και εξυπηρετητή, καθώς και υλοποιήσεις για την αποστολή-λήψη των μηνυμάτων και λίστα πελατών, τα οποία θα πρέπει να ενημερώσουμε στην συνέχεια.

Πιο συγκεκριμένα, θα υλοποιήσουμε τρεις διαφορετικές εκδόσεις, όπου με την χρήση νημάτων θα προσθέτουμε μεγαλύτερη πολυπλοκότητα κάθε φορά, με την τελική έκδοση να υποστηρίζει πολλούς πελάτες που μπορούν να συνομιλούν ταυτόχρονα.

2 ΕΚΔΟΣΗ #1: ΑΙΤΗΣΗ-ΑΠΑΝΤΗΣΗ ΜΕ ΕΝΑ ΝΗΜΑ ΑΝΑ ΔΙΕΡΓΑΣΙΑ

Σε αυτή την εκδοχή, ο εξυπηρετητής μπορεί να στείλει απάντηση στον πελάτη, ο οποίος περιμένει μέχρι να την λάβει.

Στο αρχείο **server.c** λοιπόν μέσα στη `main` στο κύριο `while` loop, πρέπει να διαχειριστούμε την απάντηση του εξυπηρετητή κάτω από τον κώδικα για την λήψη του μηνύματος από τον πελάτη. Ορίζουμε την μεταβλητή **reply** στην οποία θα αποθηκευτεί το μήνυμα από το πληκτρολόγιο και στη συνέχεια μέσα στο `do-while` loop ο εξυπηρετητής θα στέλνει μηνύματα, παρόμοια και με τον πελάτη στην αρχική έκδοση. Με την προτροπή `>` στο τερματικό διαβάζεται το μήνυμα από το πληκτρολόγιο για όσο δεν έχει δοθεί EOF και στέλνεται στον πελάτη με **sendMessage()**.

Στο **client.c** κάτω από την αποστολή μηνύματος με **sendMessage()** ορίζουμε ξανά μεταβλητή **reply** για την απάντηση που λαμβάνει ο πελάτης από τον εξυπηρετητή. Έτσι, κάνουμε **recvMessage()** για να λάβει το μήνυμα του εξυπηρετητή και το διαχειριζόμαστε όπως και στο **server.c**. Αν ληφθούν κανονικά τα bytes λοιπόν, τυπώνεται το μήνυμα του εξυπηρετητή.

Τελικά βλέπουμε ότι όταν ο πελάτης συνδεθεί στον server, αυτός περιμένει να σταλθεί κάποιο μήνυμα και αφού γίνει αυτό μετά εμφανίζεται η προτροπή `>` για να γράψει. Το ίδιο ισχύει και για τον πελάτη. Όταν στείλει κάποιο μήνυμα μετά πρέπει να περιμένει τον server να απαντήσει, αφού η **recvMessage()** είναι blocking.

3 ΕΚΔΟΣΗ #2: ΑΙΤΗΣΗ-ΑΠΑΝΤΗΣΗ ΜΕ ΔΥΟ ΝΗΜΑΤΑ ΑΝΑ ΔΙΕΡΓΑΣΙΑ

Εδώ επεκτείνουμε την προηγούμενη έκδοση και προσθέτουμε ένα επιπλέον νήμα στον εξυπηρετητή και στον πελάτη, ώστε να μην περιμένουν κάθε φορά την λήψη απάντησης.

Στο **client.c** στη **main** αφήνουμε όπως είναι τον κώδικα για την αποστολή μηνύματος και μετακινούμε το κομμάτι του **receive** στη νέα συνάρτηση **receive_msg_thread_func()**. Αυτή λαμβάνει ως όρισμα τον περιγραφέα αρχείου του socket και σε ένα while loop λαμβάνει τις απαντήσεις του εξυπηρετητή όπως πριν. Σε περίπτωση που γίνει κάποιο σφάλμα ή ο server κλείσει ή στείλει EOF, δηλαδή λάβει μηδενικά bytes, κάνουμε shutdown ώστε τερματίσουν όλα τα νήματα που περιμένουν. Στη **main** πάνω από την αποστολή μηνύματος φτιάχνουμε με **pthread_create()** το νέο νήμα που θα καλέσει αυτή τη συνάρτηση και έξω από τον βρόγχο όπου ο πελάτης θα έχει πια σταματήσει, καλούμε shutdown για να ενημερωθεί το άλλο νήμα και **pthread_join()** ώστε να περιμένουμε το νήμα να τελειώσει. Για να τερματίσει ομαλά ο πελάτης αν ο εξυπηρετητής κλείσει, ορίσαμε την μεταβλητή **server_disconnected** που ενημερώνεται κατάλληλα όταν δεν ληφθούν σωστά bytes με την **recvMessage()**. Χρησιμοποιούμε και την **temp_disconnect_flag** για να τερματίσει ομαλά και το νήμα της **main** και φυσικά χρειαζόμαστε την κλειδαριά **disconnect_mutex** για να μην υπάρξει πρόβλημα συγχρονισμού μεταξύ των δύο νημάτων. Τέλος, επειδή η **recvMessage()** επιστρέφει 0 bytes όταν τερματίζει ο χρήστης και ο server, ορίζουμε την μεταβλητή **user_quits** για να ξεχωρίσουμε τα μηνύματα που εμφανίζονται σε περίπτωση αποσύνδεσης. Η μεταβλητή γίνεται 1 όταν ο πελάτης πατήσει Ctrl+D. Την ενημερώνουμε πάντα μέσα στο mutex αφού την βλέπουν και τα δύο νήματα και μέσα στην συνάρτηση του νήματος την αντιγράφουμε με ασφάλεια στην **tmp_user_quits**.

Στο **server.c** αφήνουμε όπως είναι στην **main** την αποστολή μηνύματος προς τον πελάτη και μετακινούμε την λήψη στη νέα συνάρτηση **receive_msg_thread_func()**, την οποία θα εκτελέσει το νέο νήμα. Η συνάρτηση διαχειρίζεται την λήψη μηνύματος από τον πελάτη όπως και πριν και σε περίπτωση σφάλματος όπως αν στείλει EOF ο client, καλεί shutdown για τον ίδιο λόγο που εξηγήσαμε πριν. Ορίσαμε και το struct **thread_args** για να περάσουμε της πληροφορίες του πελάτη στο νήμα. Στη **main** πάνω από το κομμάτι του **sendMessage()**, δεσμεύουμε χώρο στη μνήμη για το struct και δημιουργούμε το νέο νήμα. Τέλος, κάνουμε ξανά shutdown και **pthread_join**. Και εδώ για να τερματίζει ομαλά ο εξυπηρετητής σε περίπτωση που αποσυνδεθεί ο πελάτης, ορίζουμε την **client_disconnected** και την διαχειριζόμαστε παρόμοια με την αντίστοιχη μεταβλητή του πελάτη. Επειδή σε αυτή την έκδοση μπορεί και ο εξυπηρετητής να σταματήσει με Ctrl+D, για τους ίδιους λόγους με τον πελάτη, ορίζουμε την μεταβλητή **server_quit**.

Αυτή τη φορά βλέπουμε κατά την εκτέλεση ότι υπάρχει πάντα προτροπή ">" για τον πελάτη και τον εξυπηρετητή στο τερματικό και μπορούν να στείλουν μήνυμα ανεξάρτητα από το πότε θα λάβουν απάντηση.

4 ΕΚΔΟΣΗ #3: ΑΙΤΗΣΗ ΧΩΡΙΣ ΑΠΑΝΤΗΣΗ ΜΕ ΠΟΛΥΝΗΜΑΤΙΚΟ ΕΠΕΞΕΡΓΑΣΤΗ

Σε αυτή την τελική έκδοση, υποστηρίζονται πολλοί πελάτες που μπορούν να συνδεθούν και να επικοινωνήσουν ταυτόχρονα. Η συγκεκριμένη έκδοση χωρίζεται σε πολλά επιμέρους βήματα τα οποία αναλύουμε παρακάτω.

4.1 ΑΛΛΑΓΗ ΔΟΜΗΣ ΕΞΥΠΗΡΕΤΗ

Θα αλλάξουμε την δομή μέσα στην **main()**. Αυτό που θα κάνουμε είναι να μετακινήσουμε την **accept** μέσα στο while loop, προκειμένου να μπορεί να δέχεται και άλλες συνδέσεις. Στην περίπτωση που προκύψει κάποιο λάθος, απλά θα αγνοήσει αυτή την προσπάθεια σύνδεσης. Αν όλα πάνε καλά, ξεκινάει η διαδικασία προσθήκης του χρήστη στην εφαρμογή. Αρχικά, το πρώτο που κάνουμε είναι να προσθέσουμε τον νέο χρήστη στην δομή που αποθηκεύουμε τους συνδεδεμένους χρήστες με την χρήση της συνάρτησης **addClient()**.

Αμέσως μετά δεσμεύουμε την απαιτούμενη μνήμη για το struct που δημιουργήσαμε για να κρατάει τις απαιτούμενες πληροφορίες για κάθε χρήστη, οι οποίες είναι το file descriptor του και το όνομά του. Σε περίπτωση που κάτι πάει λάθος κατά την δέσμευση μνήμης, αφαιρεί τον χρήστη από την λίστα και τερματίζει την σύνδεση του. Διαφορετικά, δημιουργεί ένα έξτρα νήμα το οποίο θα απασχολείται με την αποστολή μηνυμάτων του συγκεκριμένου χρήστη. Στο νήμα περνάμε και ένα προκαθορισμένο όνομα για τον χρήστη μέσω του struct το οποίο ενημερώνεται με το **client_counter**. Αν κάτι πάει λάθος κατά την δημιουργία του νήματος, αφαιρεί τον χρήστη από την λίστα και τερματίζει την σύνδεση του, αλλιώς κάνει detach το νήμα προκειμένου να τρέχει ανεξάρτητα. Στο τέλος του προγράμματος κάνουμε **destroyClientsList()**.

Η συνάρτηση την οποία τρέχει το νέο νήμα είναι η **handle_client()**. Σε αυτήν, αρχικά ενημερώνουμε πως συνδέθηκε ο νέος χρήστης. Έπειτα μέσα στο while loop με την χρήση της συνάρτησης **recvMessage()**, μόλις ληφθεί το μήνυμα από τον server, αρχικά ελέγχουμε αν υπάρχει κάποιο σφάλμα, προκειμένου να το διαχειριστούμε κατάλληλα. Αν όλα είναι εντάξει, ελέγχουμε να δούμε αν πρόκειται για μήνυμα/εντολή αλλαγής ονόματος όπου αλλάζουμε το όνομα του χρήστη, σε διαφορετική περίπτωση απλά στέλνουμε σε όλους τους υπόλοιπους χρήστες με την χρήση της συνάρτησης **broadcastClientsList()** και κάνουμε print και στην κονσόλα του server ποιος συνδέεται/αποσυνδέεται καθώς και αλλαγές ονομάτων.

4.2 ΕΚΠΟΜΠΗ ΜΗΝΥΜΑΤΟΣ

Προκειμένου να στέλνονται σωστά τα μηνύματα, είναι απαραίτητες οι αλλαγές και σε πολλά άλλα αρχεία. Θα ξεκινήσουμε από το αρχείο **client.c**. Σε αυτό το αρχείο, αυτό που κάνουμε πριν το μεγάλο do-while loop είναι να δημιουργούμε ένα νήμα το οποίο είναι υπεύθυνο για την λήψη μηνυμάτων. Το μεγάλο do-while loop τρέχει για όσο η μεταβλητή EOF_FOUND είναι 0 (το οποίο αλλάζει όταν ο χρήστης πατήσει ctrl+D). Το εσωτερικό do-while loop τρέχει για όσο ο χρήστης δεν πατάει enter ή μέχρι να πατήσει ctrl+D. Μόλις βγει από αυτό το loop ελέγχουμε αν ο χρήστης πάτησε ctrl+D, όπου σε αυτή την περίπτωση δεν στέλνει κάποιο μήνυμα και απλά τερματίζει. Σε διαφορετική περίπτωση, στέλνει το μήνυμα του με την **sendMessage()** και αν κάτι πάει λάθος κατά την αποστολή, τότε τερματίζει την σύνδεση. Εδώ δεν χρειάζεται να στείλουμε και το όνομα (με την **sendNewMessage()**) αφού ο εξυπηρετητής είναι υπεύθυνος για να το προσθέσει. Στο τέλος του loop, καλούμε την shutdown για να ενημερωθεί και το άλλο νήμα, έπειτα καλούμε την **pthread_join()** και κλείνουμε την σύνδεση.

Η συνάρτηση που καλεί το άλλο νήμα που δημιουργούμε είναι η **receive_msg_thread_func()**. Σε αυτή, έχουμε ένα μεγάλο while loop όπου αρχικά καλούμε την **recvNewMessage()** προκειμένου να λάβουμε το μήνυμα με το όνομα του αποστολέα. Αμέσως μετά, ελέγχουμε για κάποιο πιθανό λάθος κατά την λήψη, όπου και εκτυπώνουμε ένα διαγνωστικό μήνυμα και τερματίζουμε την σύνδεση μας. Αλλιώς, απλά κάνουμε print το μήνυμα που λάβαμε. Επειδή η **recvNewMessage()** επιστρέφει 0 bytes και στην περίπτωση που τερματίσει ο πελάτης και στην περίπτωση που κλείσει ο server, ορίσαμε την μεταβλητή **user_quit**, η οποία χρησιμοποιείται όπως και στην προηγούμενη έκδοση. Γίνεται 1 όταν ο πελάτης πατήσει Ctrl+D και την χρησιμοποιούμε για να διαφοροποιήσουμε τα ενημερωτικά μηνύματα σε περίπτωση αποσύνδεσης client/server.

Οι επόμενες αλλαγές που κάναμε ώστε να στέλνονται και να λαμβάνονται τα μηνύματα μαζί με το όνομα του κάθε πελάτη, είναι στο αρχείο **msg.c**. Η πρώτη συνάρτηση που χρειάστηκε να τροποποιήσουμε είναι η **sendNewMessage()**. Αυτό που κάνουμε σε αυτήν, είναι να στέλνουμε πρώτα το μήκος του ονόματος, έπειτα το όνομα και τέλος το μήνυμα. Τα πρώτα δύο τα στέλνουμε με την **writeall()** ενώ το κύριο μήνυμα με την **sendMessage()**.

Η επόμενη συνάρτηση είναι η **recvNewMessage()** στην οποία πρώτα διαβάζουμε το μήκος του ονόματος, έπειτα το όνομα και τέλος το κύριο μήνυμα. Τα πρώτα δύο τα διαβάζουμε με την **readall()** ενώ το κύριο μήνυμα με την **recvMessage()**.

Έπειτα, συνεχίζουμε στο αρχείο **clientsList.c**. Η συνάρτηση που θα υλοποιήσουμε είναι η **broadcastClientsList()** ώστε ο εξυπηρετητής να προωθεί το μήνυμα σε όλους τους πελάτες. Αρχικά, το πρώτο που κάνουμε είναι να κλειδώσουμε το mutex εφόσον πρόκειται να χρησιμοποιήσουμε κοινόχρηστη μεταβλητή. Διατρέχουμε όλους τους χρήστες και για κάθε χρήστη εκτός του αποστολέα, στέλνουμε το μήνυμα με την χρήση του **sendNewMessage()**. Στο τέλος ξεκλειδώνουμε το mutex.

5 ΜΠΟΝΟΥΣ: ΔΙΑΤΗΡΗΣΗ ΙΣΤΟΡΙΚΟΥ ΣΥΝΟΜΙΛΙΑΣ

Για την υλοποίηση του bonus, βασιστήκαμε στην τρίτη έκδοση της εφαρμογής και την επεκτείναμε. Η αλλαγές βρίσκονται στο αρχείο **server.c**. Αρχικά, δημιουργούμε ένα struct που ονομάζουμε **history_node** και κρατάει πληροφορία για τον αποστολέα του μηνύματος, το μήνυμα και ένα struct δείκτη που δείχνει στο επόμενο μήνυμα. Επίσης δημιουργούμε δύο struct δείκτες, τους **history_head** και **history_tail**, οι οποίοι δείχνουν στην αρχή και το τέλος της λίστας μηνυμάτων αντίστοιχα. Δημιουργούμε και ένα mutex για το ιστορικό.

Το επόμενο που κάνουμε είναι να δημιουργήσουμε μία συνάρτηση, την **add_to_history()**, η οποία κάθε φορά που ο server λαμβάνει ένα μήνυμα καλείται για να προσθέσει αυτό το μήνυμα στην λίστα. Σε αυτήν, δεσμεύουμε μνήμη για το struct pointer το οποίο είναι υπεύθυνο για να αποθηκεύσουμε τις απαραίτητες πληροφορίες για το καινούργιο μήνυμα. Έπειτα αποθηκεύουμε τις πληροφορίες του μηνύματος στα αντίστοιχα πεδία και θέτουμε το πεδίο next ως null. Αυτό που κάνουμε αμέσως μετά είναι να κλειδώσουμε το mutex για το ιστορικό αφού αποκτάμε πρόσβαση σε αυτό (και είναι κοινόχρηστο αφού μπορούν να αποκτήσουν πρόσβαση και τα δύο νήματα). Ελέγχουμε αν πρόκειται για το πρώτο μήνυμα και αν ισχύει αυτό τότε και ο δείκτης του πρώτου μηνύματος και του τελευταίου της λίστας θα δείχνουν στο ίδιο μήνυμα. Διαφορετικά, τροποποιούμε μόνο το δείκτη που δείχνει στο τελευταίο μήνυμα και το πεδίο next δείχνει στο νέο μας μήνυμα ενώ θέτουμε αυτό ως τελευταίο. Τέλος ξεκλειδώνουμε το mutex.

Φυσικά, εφόσον κατά την δημιουργία των nodes για τα μηνύματα του ιστορικού χρησιμοποιούμε την malloc, χρειαζόμαστε και μία συνάρτηση η οποία θα αποδεσμεύει την μνήμη. Έτσι, δημιουργήσαμε την **free_history()**. Εκεί, αρχικά κλειδώνουμε το mutex αφού αποκτάμε πρόσβαση σε κοινόχρηστη μεταβλητή. Δημιουργούμε δύο μεταβλητές struct με τις οποίες θα διατρέχουμε την λίστα των μηνυμάτων. Έπειτα σε ένα while loop για όσο η μεταβλητή current (η οποία κάθε φορά είναι το μήνυμα που θέλουμε να αποδεσμεύσουμε) είναι διάφορη του null, αρχικά θέτουμε ως next_node το node που δείχνει κάθε φορά το επόμενο του current και έπειτα αποδεσμεύουμε την μνήμη. Αμέσως μετά θέτουμε το current ίσο με το next_node. Στο τέλος της συνάρτησης φυσικά ξεκλειδώνουμε το mutex.

Η επόμενη συνάρτηση που υλοποιούμε είναι η **send_history_to_client()**. Σε αυτήν φυσικά το πρώτο που κάνουμε είναι να κλειδώσουμε το mutex του ιστορικού. Δημιουργούμε έναν δείκτη **current** με τον οποίο θα διατρέχουμε την λίστα και για όσο αυτό είναι διάφορο του null, στέλνουμε μήνυμα στον χρήστη και προχωράμε στο επόμενο μήνυμα του ιστορικού. Στο τέλος, ξεκλειδώνουμε το mutex.

Εφόσον ολοκληρώσαμε αυτά, μένει να προσθέσουμε τις συναρτήσεις στα κατάλληλα σημεία. Αρχικά μέσα στην **handle_client()**, καλούμε την συνάρτηση **send_history_to_client()** ακριβώς πριν το while loop όπου και είναι το σημείο που μόλις έχει συνδεθεί ο νέος πελάτης. Τέλος, θα προσθέσουμε και την **add_to_history()** ακριβώς μετά την **broadcastClientsList()**.

Έτσι, κάθε φορά που συνδέεται ένας νέος πελάτης, θα βλέπει όλο το ιστορικό της συνομιλίας.